UBS VERIFICATIONS · DECISION ENGINE · JULY 2026

The Verifications Decision Engine

2.75 million replayed case-days — and the learning operation that evidence can unlock. Four pieces: the business case, the ceiling of today's automation, one case through the machine, and the engineering path from prediction to automated decisions.

1	**The Verifications Decision Engine**	The executive briefing — the business case, the specialist ML fleet, the Case Control Engine, and the evidence
2	**The ceiling of scripted automation**	The field report — the bot's measured limits, and its future as the engine's execution layer
3	**How one case moves through the machine**	The illustrated walkthrough — case 4127 today, and in the next stage
4	**How machine learning becomes the next best move**	The engineering deep dive: models, resolver, gates, text layer — and the layers that attach next

Reading notes. All figures are backtested and as-of (no future information); correlations are observational — no channel or wording is claimed to cause outcomes until the pilot measures it; the system is decision support today — nothing auto-sends yet; automation is designed in as a later, evidence-gated stage. The planned next layers: an AI contact researcher that finds fresh, verified contact paths, and a small language model that drafts the emails, faxes, notes, and call scripts — sequenced last, behind the causal instrumentation that can grade them.

SNH *A Wright Original* · Built by **SNH Strategic Initiatives**

HOW TO READ THIS PACKET

The argument in four parts

1	**The executive briefing**	establishes the business problem, the working foundation, and the destination
2	**The Murphy Brown field report**	shows why scripted execution needs a Case Control Engine around it
3	**The illustrated walkthrough**	follows one case through that layer — today, and in the next stage
4	**The engineering deep dive**	shows how the system is built, tested, and extended

One sentence carries all four: the specialist ML fleet reads the work, the Case Control Engine turns forecasts into a plan, the live record teaches it what works, and routine decisions earn automation one proven lane at a time.

SNH *A Wright Original* · Built by **SNH Strategic Initiatives**

EXECUTIVE BRIEFING

The Verifications Decision Engine

Specialist machine-learning models read where each open search is heading. The Verifications Decision Engine combines those forecasts with the current evidence to build a case-by-case plan: what to work first, when to change paths, and when to stop.

The operation has two lanes — and one missing layer

Verifications currently operates through two lanes: people working queues and automation executing individual touches. Both produce real work. Neither is supported by a common decision system that reads the case, explains what matters, and governs what should happen next.

In the manual lane, people receive a queue — not a strategy. The worklist does not consistently tell them which cases are most recoverable, which are beginning to stall, which have already received a reply, or why one case should be worked before another. Agents bring their own experience and judgment, but the system makes them reconstruct the plan case by case. The operation therefore depends on people to supply what the workflow does not: which case to work first, what has already been tried, and when to change course.

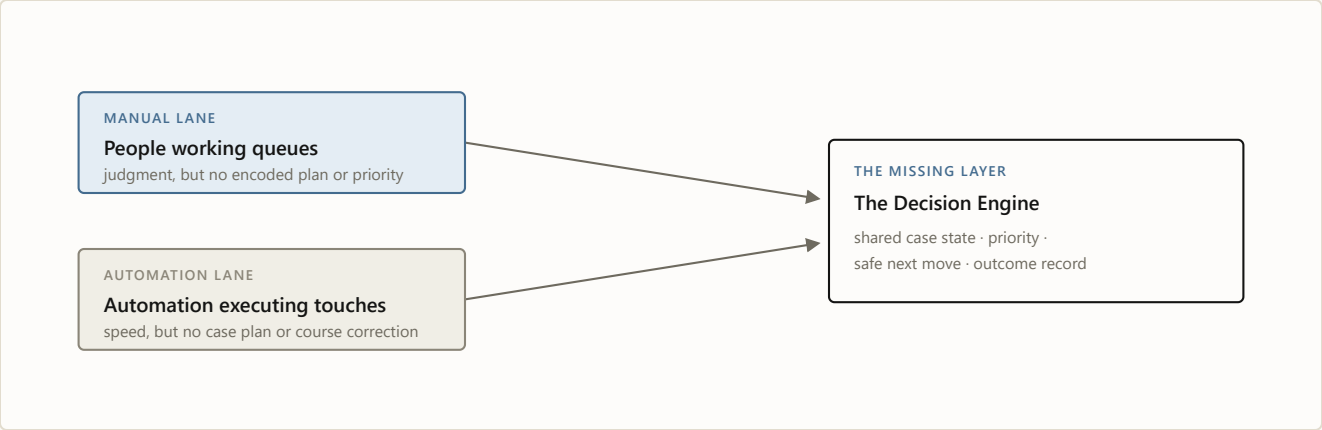
The automation lane does not work the case; it fires the script. Give it a contact plan and it sends a fixed sequence of canned emails and faxes without asking whether the case has changed, whether the last touch accomplished anything, or whether a different path now makes more sense. It makes outreach faster, not smarter. When the sequence ends, the unresolved case

returns to a person to reconstruct. *The ceiling of scripted automation*, the Murphy Brown field report, documents that pattern in detail.

Today, one in five worked cases ends "unable to verify." But that single rate combines very different cases: some were unlikely to verify from the start, some were blocked by missing or weak contact information, and some were recoverable but stalled in the workflow. Because all three disappear into the same number, leadership can see how often verification failed — but not why, which misses were avoidable, or where a different next move could have changed the outcome.

The common problem is not a lack of effort or automation. It is that neither lane has a system that reads the whole case, creates the plan, governs the next move, and records what happened. People must supply the plan; automation only executes the script. Neither consistently produces the complete record needed to learn what works.

That is the problem the Verifications Decision Engine was built to solve.



Two lanes. One missing layer.

The original ask was a learning operation, not just a model

The original brief contained two connected ambitions: identify early which searches are likely to verify within five days or ultimately end unable to verify, and learn the complete contact play — channel, timing, cadence, and language — well enough to automate it.

Both depend on the same foundation: a complete record of what the case looked like, what the system recommended, what happened next, what was sent, what came back, and how the case ended.

The specialist models already produce five-day and final-UTV forecasts in historical replay. The Case Control Engine turns those forecasts into a case plan. Live integration creates the outcome record needed to improve that plan, learn which communication works, and automate the routine decisions that prove themselves.

The specialist prediction models used by the current engine are not the future SLM. They estimate where each search is heading. The Case Control Engine uses those forecasts to build the plan the future SLM will write against and the outcome record that will judge its work. The first specialist forecasts were working within one week. That demonstrates how quickly useful machine learning can be built on our casework; the language layer remains the next phase.

What we built: intelligence plus control

MACHINE-LEARNED INTELLIGENCE

a fleet of specialist machine-learning models · reads every open case each morning

Different specialists estimate different parts of the case's path: whether it is likely to resolve quickly, whether it is beginning to stall, and whether it is likely to verify at all.

The models do not authorize actions. They provide an informed read of where the case is heading and which eligible move deserves attention first.

THE CASE CONTROL ENGINE

rules name state and remove · model scores order · a person decides

The Case Control Engine turns those forecasts into a working case plan. It names the case's current state, determines which moves fit, uses the model scores to order the choices, and presents one recommendation to a verification specialist.

The models answer what is likely. The Case Control Engine determines how that intelligence becomes a workable plan. A verification specialist decides what happens now.

WHAT THE FORECASTS LOOK LIKE

Case 4127 ILLUSTRATIVE

68% chance of verification within five days · 14% final-UTV risk · next move: email today · if no reply: call after the measured response window.

The Verifications Decision Engine is the whole system. The Case Control Engine is the resolver, scorer, router, and outcome record that turns the specialist models' forecasts into a workable case plan. Case 4127 shows the form of that output; its values are illustrative.

Every case follows the same handoff

1 · Evidence shows what is known	The morning record establishes what is currently known about the case.

2 · The model fleet predicts where it is heading	Specialists estimate where the case is heading — resolve quickly, begin to stall, or never verify.
3 · Rules name the state what is true now	The Case Control Engine decides what is true now — replied, blocked, cooling down, workable.
4 · Rules remove; models rank eligible moves, ordered	Prohibited or premature actions leave the table; the eligible choices that remain are ordered.
5 · A person decides accept or override	The recommendation is accepted or overridden — and the override is itself a signal.
6 · The outcome teaches into the record	With live integration, the decision, communication, reply, and final result will enter the learning record.

The model never receives authority to restore an action the rules removed. Prediction improves the work while permission stays with the rules.

What 2.75 million historical case-days proved

The historical replay evaluated one daily record for every open case on every day it remained open — 2.75 million case-days across the year. Four findings changed how the operation can be managed.

The operation can see difficulty before spending the effort. At intake, when the system compares a case that will eventually fail with one that will verify, it ranks the future failure as harder approximately 81% of the time. Its forecasts continue separating harder from easier cases even after the obvious successes leave the queue.

One UTV rate hides cases with radically different odds. The easiest predicted tenth of cases fails about 3% of the time. The hardest tenth fails about 48% of the time. That 16-fold spread makes a fairer operating measure possible: evaluate outcomes against what was realistically achievable and direct effort toward cases where intervention can still matter.

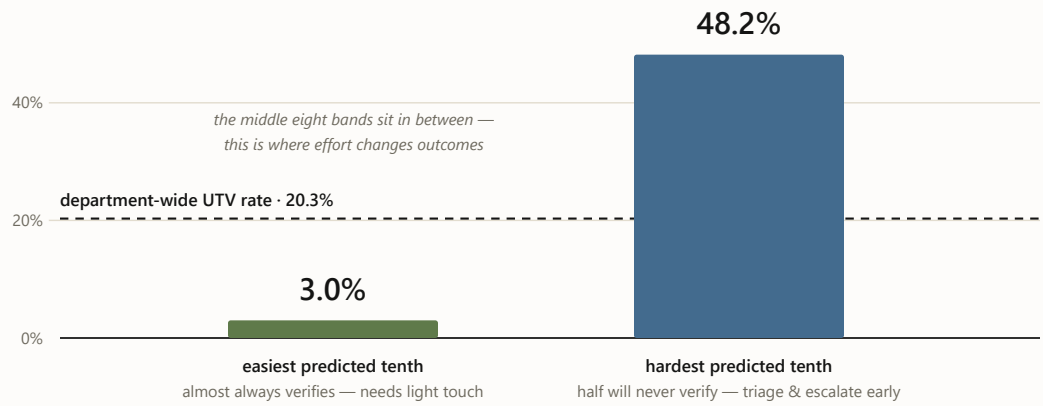


CHART 3 Observed never-verify rate by predicted difficulty band (May 2026 holdout): the easiest tenth 3.0%, the hardest 48.2% — the 16-fold spread that makes a difficulty-adjusted scoreboard possible.

A simple rule found substantial avoidable work. On roughly 400,000 replayed case-days, a response was already on record while the existing flow would have continued contacting the party. The Case Control Engine routes those cases to review before another touch can occur. At an illustrative three minutes per avoided touch, that measured count represents about 20,000 hours, or ten operator-years, of motion. The touch count is measured; the three-minute assumption is not. Live logs replace the assumption with actual handle time.

The two-part design held its safety boundary. Across 2.75 million historical case-days, the replay produced zero do-not-contact and hard-policy violations. That result did not depend on the models being perfect. The models never received authority to override the rules.

These are historical replay results, not live-pilot outcomes. They establish that the system can read, route, and constrain the work. Integration establishes the live operational effect.

For people, the queue becomes a plan

On the first day of integration:

- Every open case has a named state, reason, and next move.
- Worklists reflect difficulty and urgency rather than arrival order alone.

- A reply on record prevents another outreach attempt.
- A prohibited action is removed before ranking.
- A person can accept or override the recommendation.
- Every decision begins contributing to the learning record.

Every case should produce two outputs

The first output is operational: what to work, which path to use, how long to wait, and when to stop. The second is evidence: what was recommended, what was actually done, which channel and message were used, what came back, and how the case ended.

Today, historical replay supports the first output. Live integration creates the second. Once both exist, the system can improve the forecast, the case plan, the timing, and eventually the message. The 10,000th case should not merely be processed faster than the first. After the next validated retraining cycle, it should benefit from what the earlier cases taught the system.

Continual learning does not mean the system rewrites itself after every email. Every completed case strengthens the training record. On a controlled cadence, new challengers are trained, compared with the current models, and promoted only when they produce a better operating result. The record grows continuously; the system changes deliberately — and no lane moves to auto-send until its own logged record proves it safe, stable, and effective.

The system improves in layers. First, it learns which cases are likely to verify and which are beginning to stall. Then it learns which contact path and next move work best for comparable cases. With complete live records, it can learn channel, timing, cadence, and message together — the AI contact researcher repairing dead paths and the SLM drafting the outreach as those lanes earn it. As individual lanes produce stable results, the Case Control Engine can automate those routine decisions and connect them to execution. The same architecture can extend to new customers, recalibrated to each customer's own outcomes.

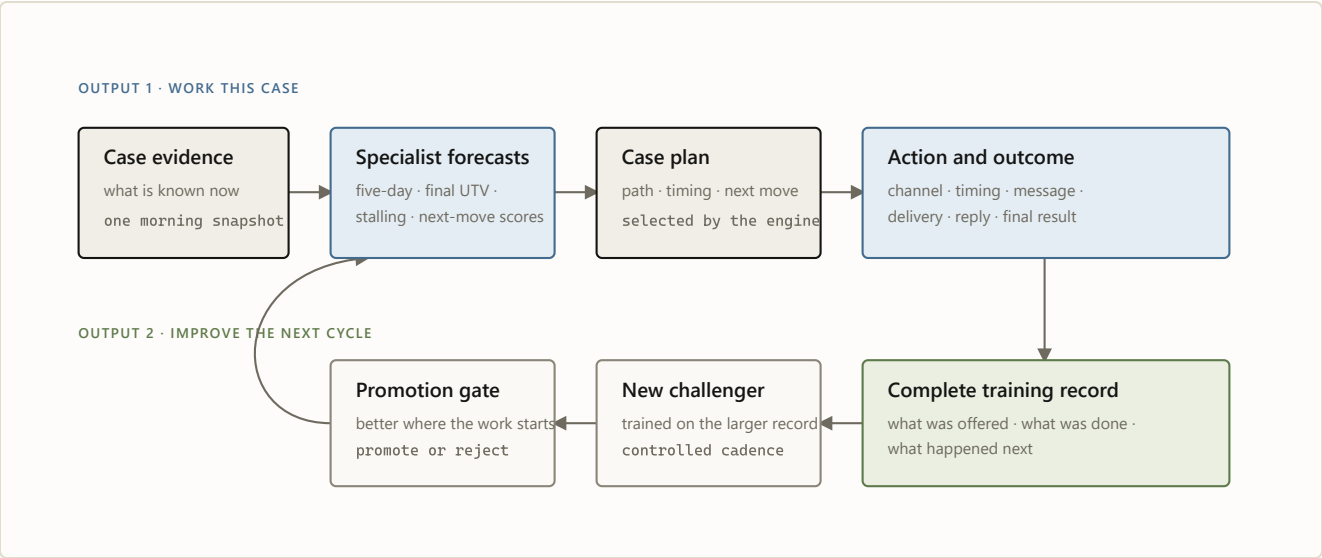


CHART 4 Every case does two jobs. The top row produces the next move. After live integration, the bottom row turns the outcome into a complete training record. The record grows continuously; the models change only when a challenger clears the promotion gate.

What leadership is being asked to do

- **Adopt difficulty-adjusted UTV as the operating scoreboard.** Measure misses against what was realistically achievable, not only against the overall department UTV rate.
- **Integrate the Decision Engine beside the live team with complete outcome logging.** Recommendations remain under human review while the operation measures adoption, overrides, turnaround, and outcomes.
- **Use the resulting evidence to sequence the larger AI build.** Contact research, SLM drafting, and controlled automation advance only from the record produced by live casework.

The immediate decision is not whether to let AI run verifications autonomously. It is whether to put the Verifications Decision Engine and outcome record into the workflow so the operation can improve now — and so routine decisions can earn automation from proof rather than promise.

That is how the original SLM ambition becomes viable: not language first, but a system that knows what it is automating and whether it worked. ◆

The fine print, kept honest. All results are historical replay results using only information available as of each case-day; no live-pilot effect is claimed. The intake and difficulty figures are translated from the validated model metrics documented in the Engineering deep dive. Historical channel and message patterns remain observational until live outcome logging captures recommendation, action or override, message version, delivery or reply, and final outcome. Nothing auto-sends or finalizes a decision in production today.

Deeper reading: the Murphy Brown field report (the problem this system closes) · the illustrated walkthrough (follow one case through the machine) · the engineering deep dive · the design philosophy.

A Wright Original · Built by **SNH Strategic Initiatives**

UBS Verifications · Decision Engine · Executive briefing · July 2026

A Wright Original · Built by **SNH Strategic Initiatives**

FIELD REPORT

The ceiling of scripted automation

Murphy Brown, our verification bot, is genuinely fast when the contact path is already clear — it fully contained about 44.5% of the tens of thousands of cases it touched over four months. The other half flowed back to people with nothing but a note, and its closures are reopening at ten times the winter rate. What the ops deep-dive actually found, why the answer is to govern the bot rather than retire it — and how it becomes the execution layer inside the decision engine.

Evidence window Mar 1 – Jun 30, 2026 (live warehouse refresh)

Claim posture association language only — see the fine print

THE 60-SECOND VERSION

- **What works:** the bot fully contains ~44.5% of what it touches, usually within a day or two. Where the contact path is already laid and the case fits the script, it executes fast — genuinely.
- **What breaks:** the other 55.5% flows back to people as note-only handoffs; the closures it does make are reopening at ten-plus times the winter rate; and its handbacks queue behind fresh work (EDUV +30.6h).
- **Why it happens:** nothing gates what it picks up, nothing owns the state it leaves behind, nothing locks what it closes, and nothing records enough to prove any of it — so people became the bot's control system. Not bugs: the ceiling of scripted automation.
- **What to do:** keep the bot; add the Case Control Engine. Five asks — closure lock, triage gate, handback re-rank, one holdout config rule (the causal number lands ~July 15 either way), and the structured payload. The bot becomes an execution channel inside that system.

Five numbers tell the story:

55.5%

Of bot-touched cases flow back to people

Roughly 14,800 of 26,700 — about four downstream human attempts apiece, 64,000+ rows in all.

~43%

Of the bot's closures now reopen after the result is sent

Up from ~3–4% in February. June figures are floors, not finals.

+30.6h

Extra wait for an EDUV handback vs. fresh work

Not a parking defect — 99.98% are agent-assigned. A queue-ranking problem.

1.7%

Containment in the worst tenth of what it picks up

Its wins were predictable before pickup (77% sorting power) — nobody was gating the door.

~45%

Of eligible tickets picked up by neither automation

About 113,000 EMPV/EDUV tickets where automation is live — the gate fails in both directions.

Source: the consolidated MB/SNH-AI findings (2026-06-30) and the July 2–3 forensics one-pager, read-only warehouse analysis. All figures are the current numbers of record for their stated windows.

A two-speed workflow

When MB touches a case, one of two things happens. About 44.5% of the time it fully contains the case — closed in one or two days, no human ever needed, genuinely fast. The other 55.5% of the time the case flows back to a person. That split, not any single defect, is the operating reality:

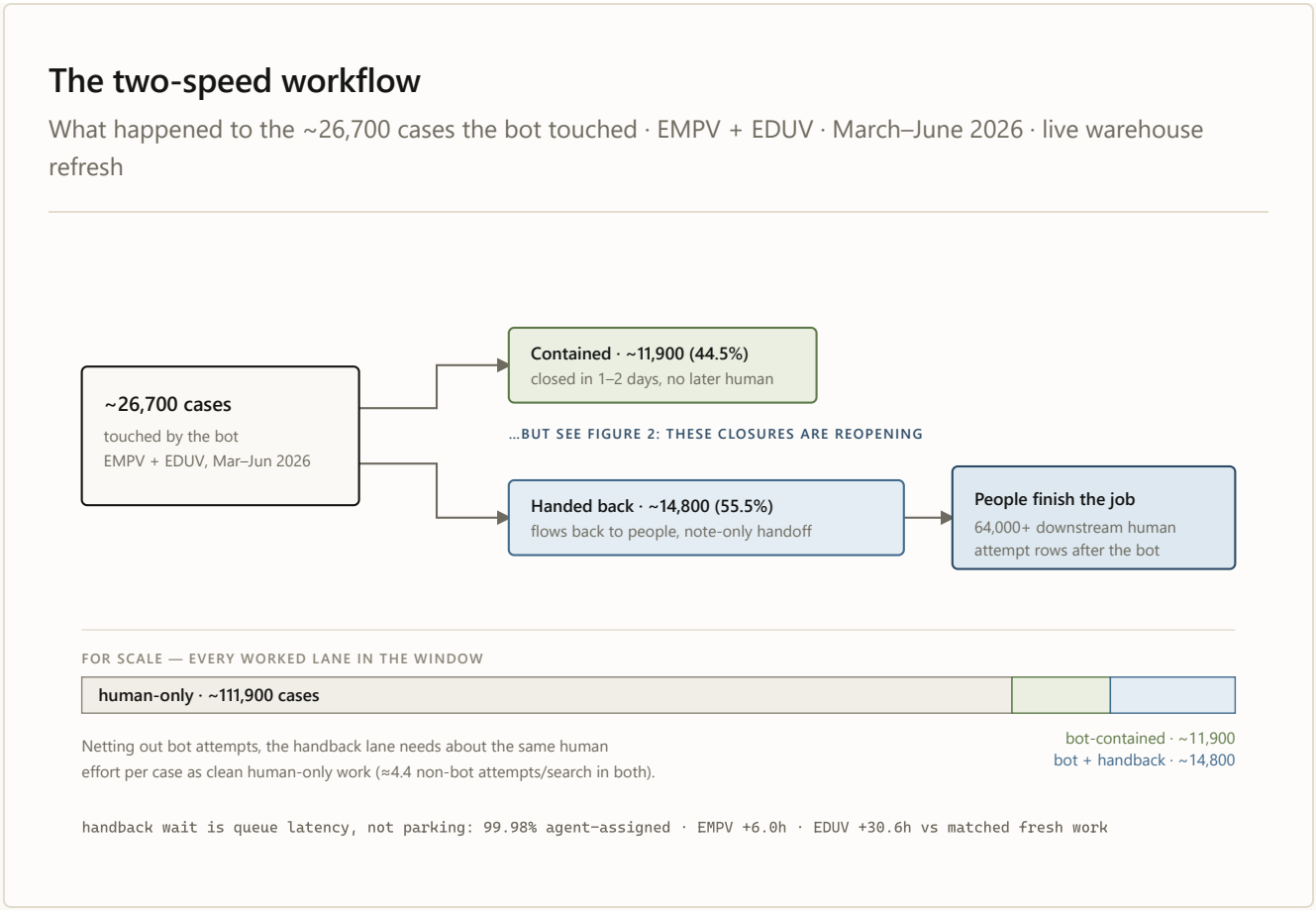


FIGURE 1 The two-speed workflow. The matched comparison is honest here: handback cases don't take clearly *more* human effort than comparable human-only cases — the durable finding is that the bot leaves a large residual queue with weak fast-closeout performance and no machine-readable handoff, so people must rediscover each case before continuing it.

Fairness requires the other half: **when the bot contains a case, it is genuinely fast.** The contained lane completes in about 61 hours versus 161 for human-only work, with a higher raw success rate (80.5% vs 73.7%). Two caveats keep that honest: these are raw lane averages, not matched comparisons — the bot picks up easier-looking work — and the clocks flatter it, because the bot fires instantly while human work queues first. Re-anchored at first touch, EMPV's speed advantage halves to −37.6 hours and EDUV flips to 23.7 hours *slower* than fresh human work. The containment is still real — which is exactly why the recommendation is to govern the bot, not retire it.

And there is a third speed the two-lane picture can't show: **never picked up at all.** Where automation is live, 44.9% of eligible EMPV/EDUV tickets — about 113,000 — were touched by neither the bot nor the SNH-AI engine behind it (measured per ticket). Part of the mechanism is already known: all-or-nothing API routing pushed about 50,000 excluded-client searches back to

manual work, a modeled 7.2–8.6k added manual searches per month that still needs configuration history to be proof-grade. The gate fails in both directions — the bot picks up cases it can't contain (its worst decile contains 1.7%) and never reaches nearly half of what it could. One gating layer fixes both.

And the contained lane — the bot's success story — has a quality problem growing inside it:

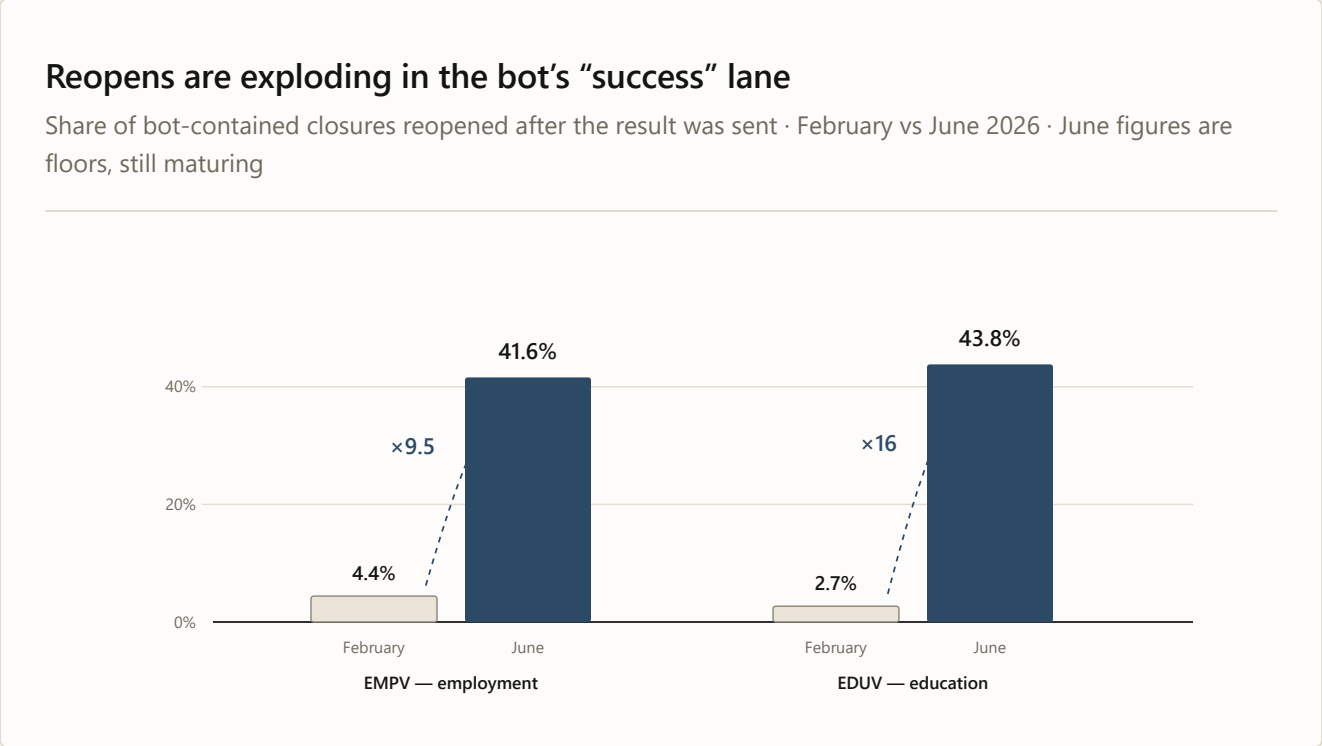


FIGURE 2 Reopen-after-result-sent on bot-contained closures, February vs. June 2026. June postdates are still maturing, so these are floors. The rise is within-lane and concentrated in bot-only closures — which is why the deep-dive's first ask is a closure lock with reopen-reason instrumentation, and why "grow containment" without one would scale a quality problem. The narrow June eForm defect (300 strict cases where the bot emailed *after* a closeout state, 1,119 manual touch rows) is the same missing lock in miniature.

A note is not a handoff

Underneath the volumes sits one mechanism, and the deep-dive names it precisely: *every strict MB touch in the diagnostic packet is note-only*. The bot does work and leaves an annotation — not a state. So the person who picks the case up must reconstruct what happened, whether evidence is complete, and what should happen next. The handoff itself creates work:

The handoff, as-is versus required

The deep-dive's core finding on the left; its required fix — the state contract — on the right

WHAT THE BOT LEAVES TODAY

"Emailed the employer contact. Will follow up."

note only · no outcome state · no next action · no owner · no retry date

WHAT A HANDOFF MUST CARRY (THE STATE CONTRACT)

`outcome_state`	completed · failed · waiting · needs-human · policy-blocked
`evidence_state`	is the case closeable right now?
`selected_contact`	which path it used — no rediscovery
`confidence`	route low-confidence work differently
`next_action`	prevents the "what now?" pickup
`owner`	who holds the case after the touch
`retry_due / cooldown`	stops same-day duplicate outreach
`pickup_reason`	separates valid continuation from rediscovery

FIGURE 3 The note versus the payload. The left side is the deep-dive's finding; the right side is its required fix — and it is, field for field, the shape of the decision engine's evidence-state and logging contracts.

The cost of that missing contract is now measured, not inferred: in the March–May window, humans **re-tried the exact channel the bot had already tried without success** — same method, same case — on roughly 6,700 cases, averaging 2–3 re-tries each (about 16,100 attempt rows), at a median five to six days after the bot's attempt. Scaled to the whole department, that is **roughly 3% of every named human attempt in the window — one touch in every thirty** — spent re-burning a channel the bot had already tried. (Method-level; per-target linkage is exactly what the payload's `selected_contact` field exists to provide.) A handoff that carried "what I tried" would have made each of those a choice instead of a rediscovery.

Every downstream defect in the issue list is the same missing contract seen from a different angle: weak cases enter ungated, channel choices are not preserved, retries lack stop rules, and closures lack a lock. Murphy Brown can execute a touch; it cannot govern the sequence around it.

Murphy Brown automates the motion, not the mission. It can send the email; it cannot run the case.

Why this is the decision engine's story to finish

The Case Control Engine supplies what the bot is missing: machine-learning models that read every case, a deterministic resolver that knows what is true and what is allowed, an action scorer that picks the play, cadence rules that run the sequence to completion, and a log that remembers what was tried. That is not an analogy — the deep-dive's fix list and the engine's component list are the same list, item for item:

Their fix list is our component list

Each measured gap, the control that closes it, and who builds it

MEASURED GAP	REQUIRED CONTROL	OWNER
Closures reopening (~43%)	Closure lock + reopen-reason codes	SNH-AI
Poor-fit pickup (worst decile contains 1.7%)	Score-gated triage eligibility	Joint
Handbacks wait +30.6h	Urgency re-ranking of returns	UBS ops
Unprovable impact	Reversible two-week holdout (~July 15)	SNH-AI
Note-only handoffs (55.5% flow back)	Structured state payload + inbound capture	SNH-AI

FIGURE 4 One list does both jobs: the deep-dive's measured gaps mapped to the decision engine's controls, with the owner of each. The engine-side controls are built and backtested; the bot-side items are the asks, and the field spec for the payload is ready to hand over.

So the proposal is not "turn Murphy Brown off." It is: **Murphy Brown becomes a channel inside the decision engine.** The resolver decides *when* the bot may touch a case; the difficulty scores decide *whether it should* (fax-primary and thin-contact cases skip it — the deep-dive's own modeling says gating to the top ~60–70% keeps 83–91% of the bot's wins while halving the handback lane); terminal states and the inbound-first rule *lock what's closed*; the priority queue re-ranks what comes back; and the flight recorder finally makes the bot's value a measured number instead of a debate.

What I'm careful not to claim

- **Association, not causation.** The bot is *associated with* a large downstream queue. The matched comparison does not prove it makes comparable cases worse on effort or time — the proven finding is the residual queue, the weak fast-closeout, and the missing handoff state.

- **No savings claims.** The handback lane's labor premium is small (≈$3/search). The money case is turnaround time and closure quality — 100,000+ EDUV wait-hours in the window, and reopen churn now measured at 0.69–3.4 reopens per automated closure (the leak-back band) but still unpriced in dollars. Attempt rows are not labor minutes, FTEs, or ROI.
- **No avoidable-percentage.** No share of the bot's downstream work is labeled "avoidable" — the record doesn't yet carry enough per-case evidence to make that judgment stick, so no avoidability number is quoted. The causal number comes from a two-week 50/50 holdout — one configuration rule on the bot's side, fully reversible — with a before-vs-after comparison against untouched cases running ~July 15 regardless.
- **Modeled is labeled modeled.** The missed-pickup mechanism (all-or-nothing API routing; about 50,000 exposed searches, ~7.2–8.6k modeled manual searches/month) still needs effective-dated configuration history to be proof-grade; the 44.9% neither-automation figure is measured per ticket where automation is live.
- **And one test the bot passed.** I probed whether MB fires before evidence exists: 0.0% of its touches happen while a vendor clock is open, and only 0.2–1.7% land before a vendor return. The bot's timing discipline is not the problem — its handoffs, closures, and gating are.

What the bot becomes

The same bot, two operating realities

What changes for Murphy Brown when the decision engine runs the case around it

MURPHY BROWN TODAY	MURPHY BROWN INSIDE THE ENGINE
Picks up whatever has a pre-laid contact path	The ML models pick the cases the bot is suited to win
Executes one scripted touch	Deterministic rules confirm the touch is allowed first
Leaves a note behind	Writes structured state and a next action
Returns uncertain work to the back of the queue	Handbacks re-enter ranked by risk and urgency
Cannot learn from the outcome	Every outcome becomes evidence for the next play

FIGURE 5 The same five behaviors, before and after governance. The left column is the measured present; the right column is the decision engine's existing components applied to the bot.

Inside the engine, the model fleet chooses the fit, rules guard the touch, and a person approves. Murphy Brown executes. As the evidence grows, AI can repair dead contact paths and an SLM can draft the outreach — but the bot's durable role is simpler: it becomes the engine's fastest pair of hands, and every outcome improves the next case. ◆

The series. This field report is the companion piece. The system it argues for: the executive briefing · the engineering deep dive · the design philosophy · the illustrated walkthrough.

Sources of record in the ops deep-dive workspace: `MB_MURPHY_BROWN_ALL_FINDINGS_CONSOLIDATED_20260630.md` and `outputs/exec_packet_20260702/MB_ONE_PAGER_20260702.md` (which supersedes earlier summaries; retired numbers per its `RETIRED_NUMBERS.md` are not cited here). All analysis read-only; windows as stated per figure; no client names, personnel names, or case identifiers appear in this report.

SNH *A Wright Original* · Built by **SNH Strategic Initiatives**

Meet **case 4127**: an employment verification, opened on a Monday. There is a work email and a phone number on file, no fax. Somewhere, an HR inbox is about to be asked to confirm a job history. Over the next nine days this case will be emailed twice, called once, and finally verified — and at every step, the system has to answer the same two questions: *how is this case doing?* and *what, if anything, should we do next?*

Case 4127 will not move through one model. It moves through a sequence of small decisions, each allowed to answer only one question — five components taking turns. Here is the cast, and the color code every diagram in this post uses:

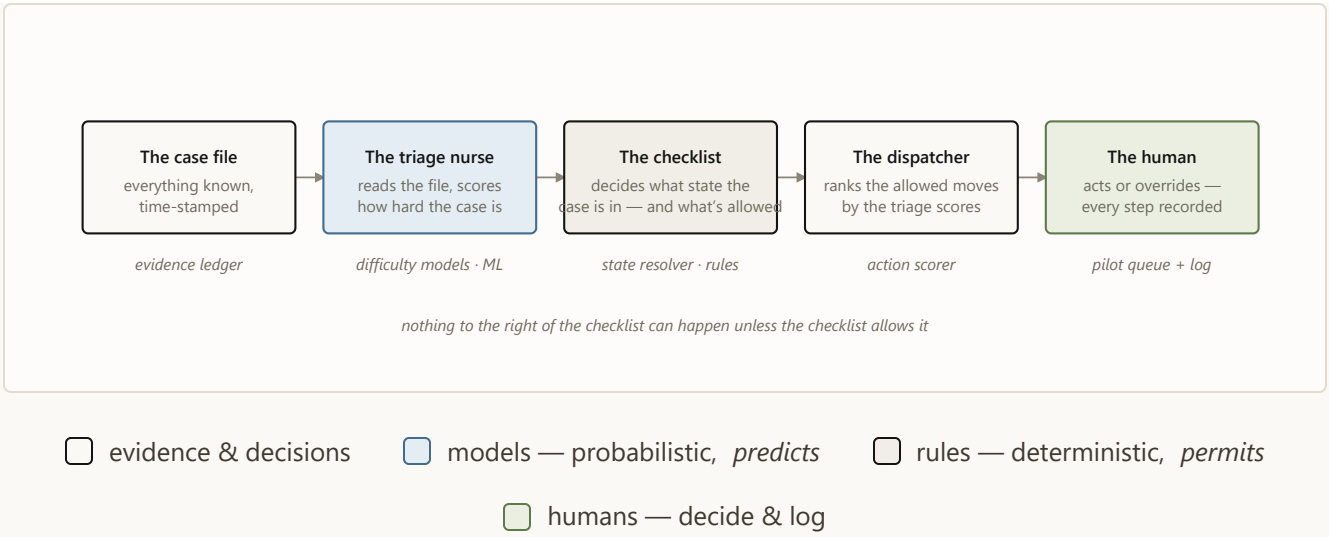


FIGURE 1 The cast, in the order they speak. The color code holds for every diagram below: blue predicts, oat permits, moss decides.

1 The curtain: what the system is allowed to know

The single most important idea in the architecture is not a model — it's a rule about time. The system never looks at a case as one blob of history. It looks at the case *as of a specific morning*, and it may only use facts from **before** that morning. One case therefore becomes many training rows — one per day it stayed open — and our twelve months of history becomes 2.75 million of these `(case, day)` snapshots.

Drag the day below and watch what case 4127 looks like from inside the system. Everything right of the curtain hasn't happened yet — as far as the machine knows, it never will.

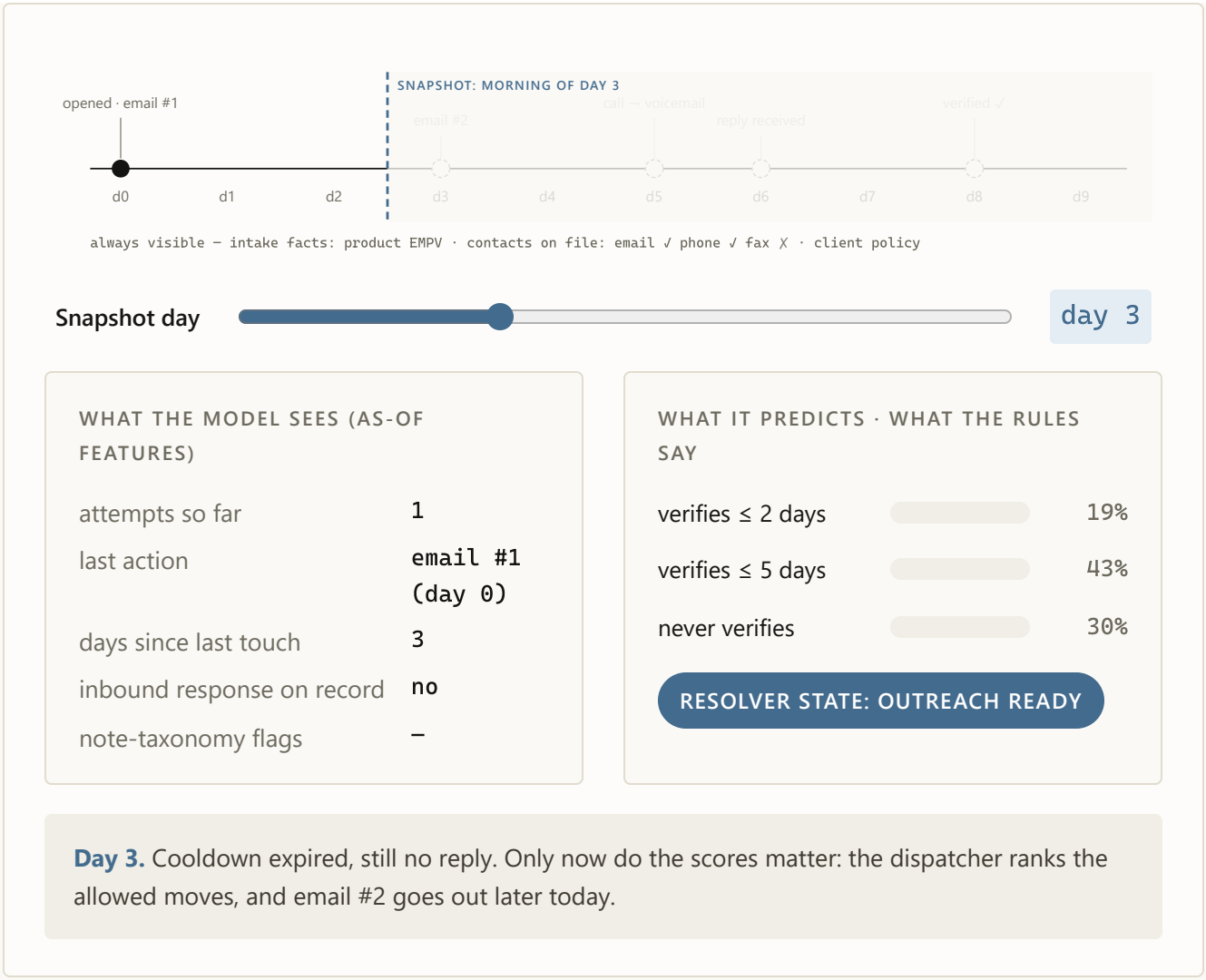


FIGURE 2 · EVIDENCE SHOWS The as-of curtain, interactively. Probabilities are illustrative. The curtain is enforced, not assumed: an automated audit re-checks it on every new training matrix before any model trains.

2 Triage: three questions about the same case

For case 4127, Monday morning — the day it arrives — is the sharpest read it will ever get, before the first attempt changes the evidence. Each morning after, three forecasting specialists from the wider model fleet re-score the visible half of the file: *will this verify within 2 days? within 5? will it ever?* Calibrated means the numbers behave like frequencies — across all cases

scored “30% never-verify,” about 30% actually never verify. That property is what makes the scores safe to build queues and thresholds on.

One case’s scores are mildly interesting. The population view is where triage earns its keep: sorted by predicted never-verify risk, the easiest tenth of cases fails 3% of the time and the hardest tenth 48%. Same process, same operators — a 16× spread in what was ever achievable. That spread powers the queue (work the winnable middle first) and a fairer scoreboard (measure misses against what was possible, not against a blended average).

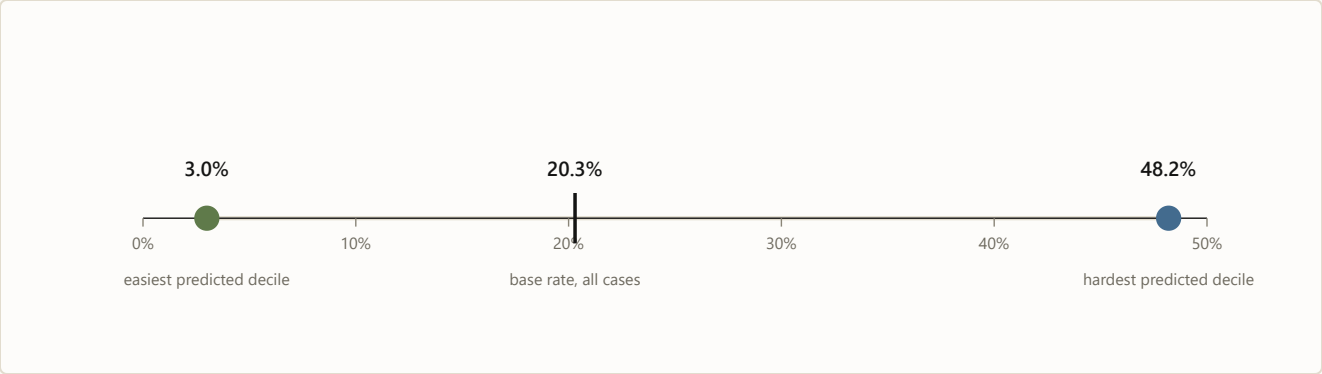


FIGURE 3 · ML PREDICTS Real numbers (May 2026 holdout): observed never-verify rate by predicted-risk decile. Triage doesn’t change any single case; it changes which cases get the attention.

3 The checklist: what state is this case actually in?

By day 3, the live question about case 4127 is no longer how hard it is — it is what is true this morning: did anyone reply? is a cooldown running? is another touch allowed? Before anyone acts on a score, a deterministic resolver asks a fixed sequence of questions any good supervisor would ask — and the first “yes” decides the case’s state for the day. The real resolver distinguishes thirteen states; the intuition fits in seven questions:

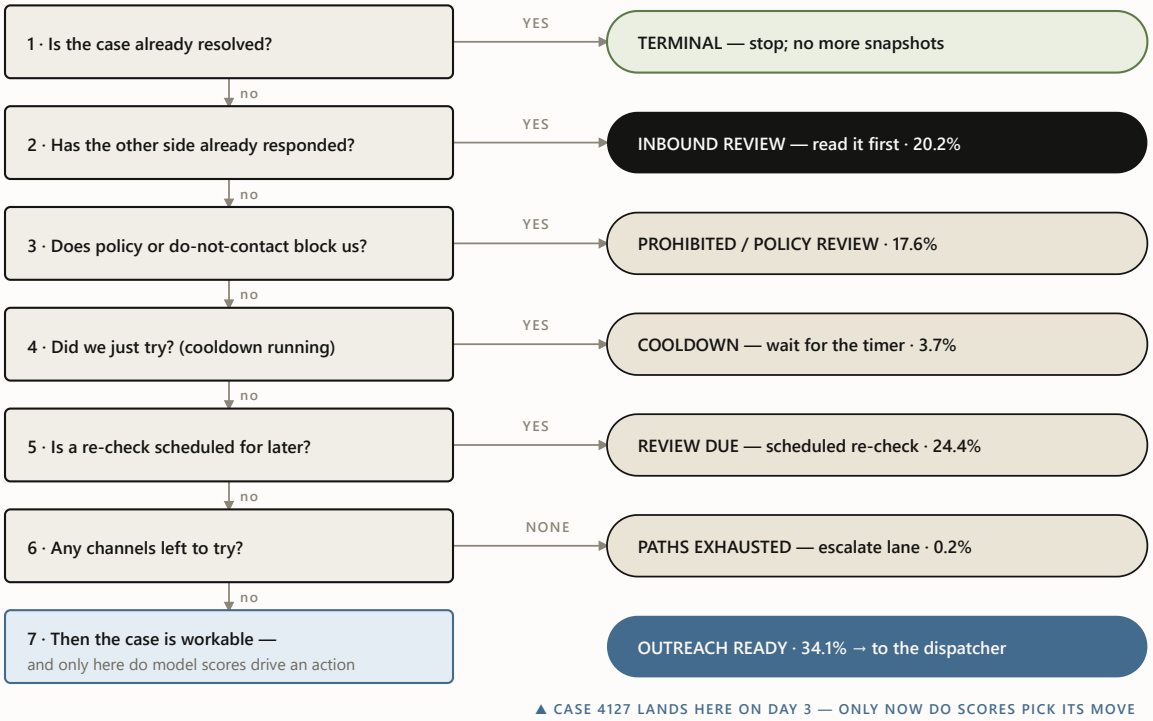


FIGURE 4 · RULES NAME The resolver as a supervisor's checklist (a seven-question simplification of the real thirteen states; percentages are each state's share of all 2.75M case-days). Notice question 2: a fifth of all case-days already have a response on record. Answering it deterministically is what caught the ~1-in-7 cases the old flow would have re-contacted. Across all 2.75M case-days: zero do-not-contact violations, zero policy violations — by construction, since scores are never consulted until row 7.

4 The bouncer and the dispatcher: six cards, one recommendation

Suppose it's day 3 and case 4127 landed on "outreach ready." The system now deals six possible moves — the same six for every case, 16.5 million candidate cards in the full run. Eligibility rules physically remove cards before any ranking happens; the model's opinion of a blocked card is irrelevant, because the card is no longer on the table.

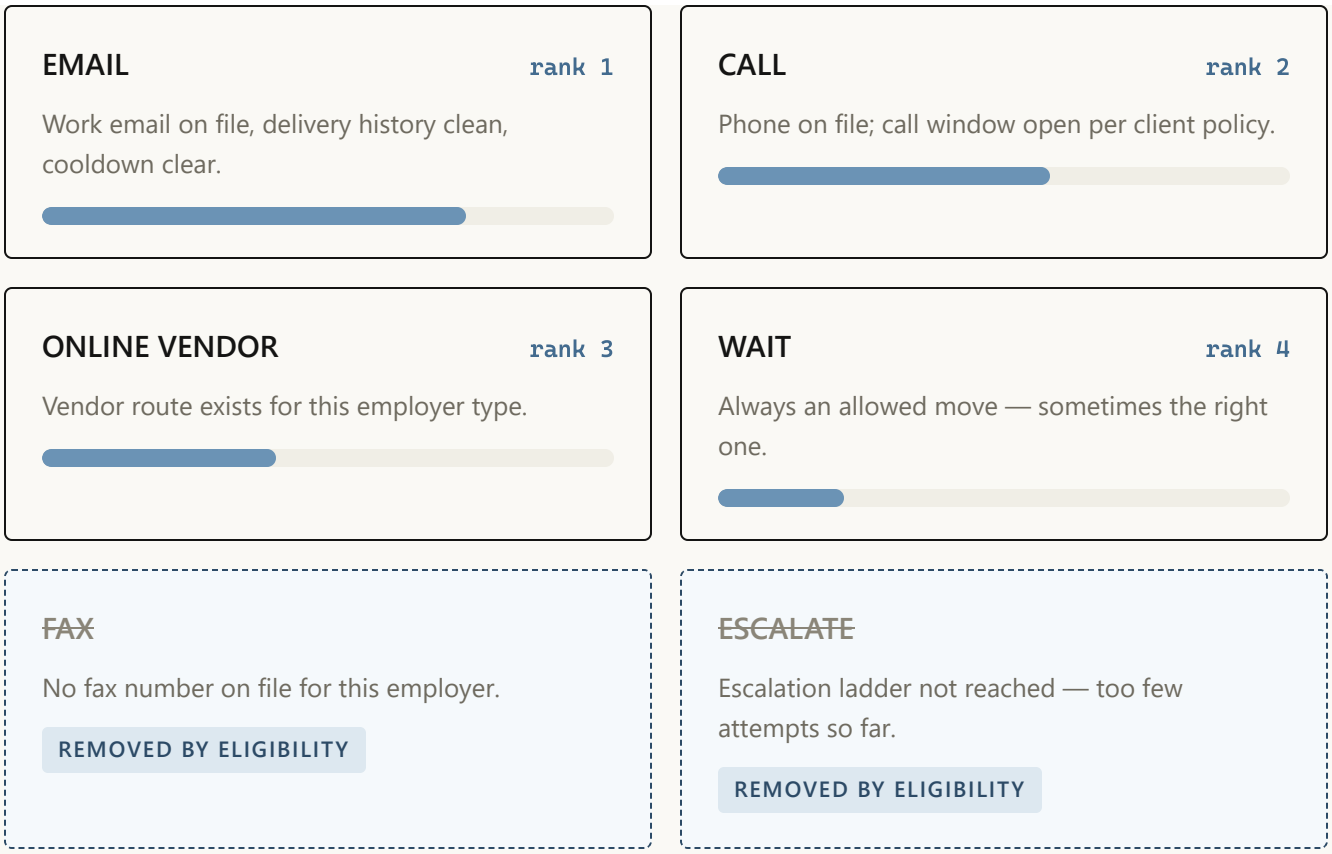


FIGURE 5 · RULES REMOVE, ML RANKS Case 4127's hand on day 3. Priority bars are illustrative; ranking reflects what historically moved similar cases under current policy — an observational signal, deliberately not a claim that any channel *causes* better outcomes. Blocked cards never reach the ranker: eligibility is a filter, not a penalty.

The dispatcher's output for the day is exactly one line: *recommended next move: EMAIL, rank 1 of 4 allowed*. In the full backtest, every single case-group ended with an available move — zero "rank-1 but blocked" contradictions, zero empty hands.

5 The flight recorder: a person decides, everything is written down

The recommendation lands in a queue in front of a person, who can take it or override it — and the override is as valuable a signal as the action. Five event types get logged, and together they form the case's complete decision story:

day 3 · 09:12	RECOMMENDED EMAIL — rank 1 of 4 allowed moves, shown to operator
day 3 · 09:14	ACTION EMAIL sent, no override · template family `follow_up` v3
day 4 · 07:40	DELIVERY delivered — no bounce
day 6 · 15:03	REPLY inbound response recorded → resolver flips to INBOUND REVIEW
day 8 · 11:26	OUTCOME verified — outcome joined back to every prior event

FIGURE 6 · PERSON DECIDES, LOG REMEMBERS Case 4127's flight record (fictional timestamps). This chain — recommendation, action or override, template version, delivery and reply, outcome — is the price of ever saying the word "because." Until it's collected at scale, the system reports correlations as observations and nothing more.

6 The heartbeat: how the machine stays honest month over month

Case 4127 verified on day 8; its complete decision story joins the next monthly evaluation cycle. New evidence rebuilds the same as-of snapshots, challengers train behind the same leakage audit, and the promotion gate records whether each specialist advances or stays put.

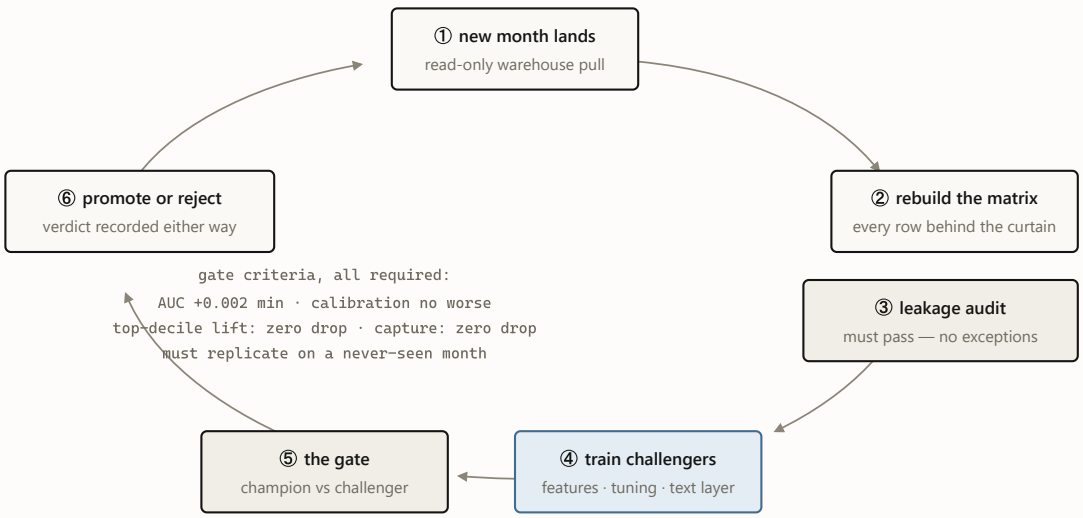


FIGURE 7 · OUTCOMES TEACH The monthly heartbeat. July 2026's cycle promoted text-aware champions for both success targets and rejected the same idea for the never-verify target — its third rejection this year, each one recorded in the champion manifest. A gate that never says no is a rubber stamp.

WHY TWO LANES AND NOT ONE BIG MODEL?

Because prediction and permission fail differently. A poor ranking wastes effort; a poor permission creates an incident. The learned lane handles the recoverable error, and the deterministic lane owns the boundary that cannot move — a single end-to-end model would blend the two and price everything at the expensive one.

The whole machine today — and what case 4127 looks like next

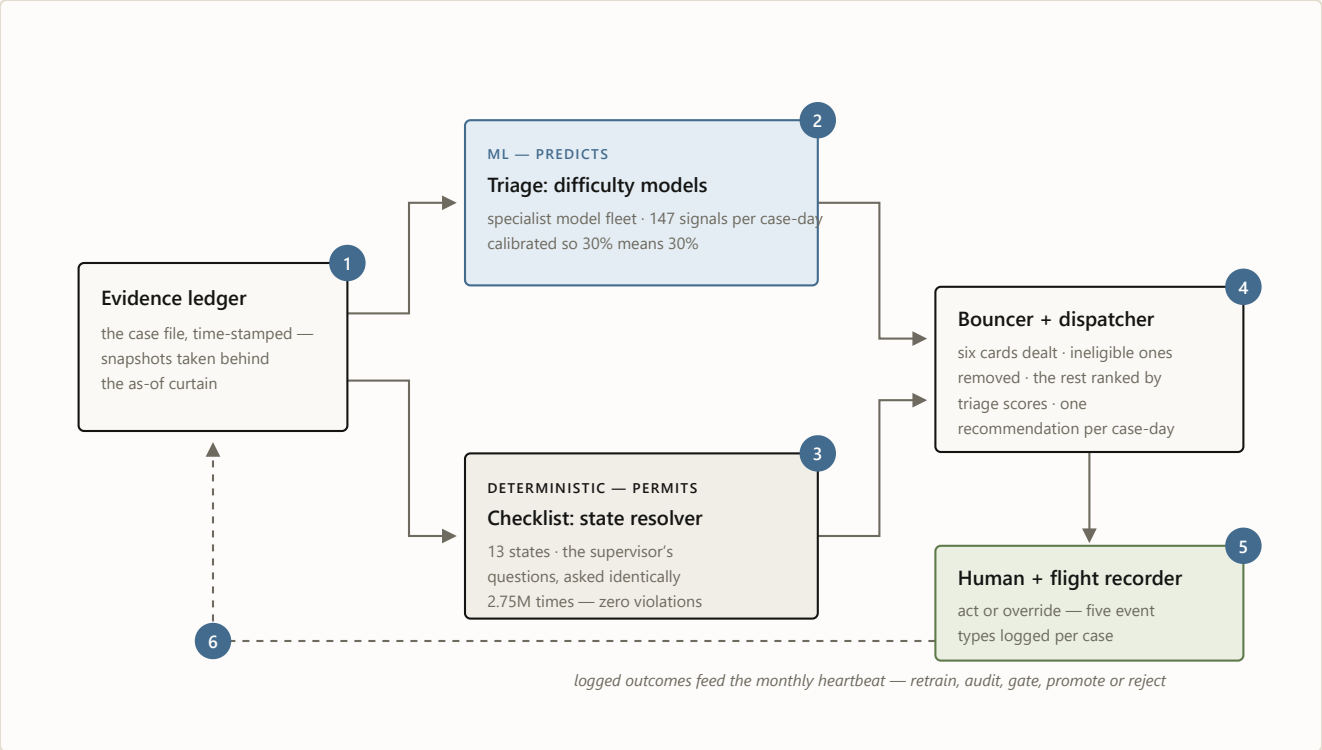


FIGURE 8 The Case Control Engine, with case 4127's six stops numbered in order: **1** the file is snapshotted behind the curtain · **2** triage scores it · **3** the checklist names its state · **4** the bouncer deals and the dispatcher ranks · **5** a person decides and the recorder writes it down · **6** the logs feed the monthly heartbeat. Blue predicts; oat permits; moss decides.

Now run the same case forward a stage. Today, the engine recognizes that case 4127 is ready for outreach, removes the blocked moves, and recommends email; a person reviews and sends it. In the next stage, the engine recommends the *complete play*: confirm the contact path is still good, email this morning with the employment-confirmation message family, and schedule a call only if no reply lands inside the measured response window. If the path on file looks dead, an AI contact researcher hunts a fresh, verified one before another attempt is spent. A small language model drafts the email; a person approves or edits it, and the system records why. And if that complete play keeps

winning across comparable cases, that one narrow lane can move to controlled auto-send on its own logged record.

The Case Control Engine does not need to change shape; its authority grows with evidence. Today it recommends the next move for a person to review. Tomorrow it learns the complete play, automates routine decisions inside proven lanes, and routes the exceptions back to verification specialists. ◆

The series: How machine learning becomes the next best move (the engineering deep dive and path to automated decisions) · this illustrated guide · plus the executive briefing and the Murphy Brown field report.

Case 4127 is fictional and its probabilities, ranks, and timestamps are illustrative. Real numbers cited: 2,751,900 case-day snapshots; 16.5M candidate actions; state-mix percentages from the 2026-06-19 resolver run; the 3.0% / 48.2% decile spread from the May 2026 holdout; zero DNC and policy violations across the full backtest. Correlations are observational — no channel or wording is claimed to cause outcomes.

SNH *A Wright Original* · Built by **SNH Strategic Initiatives**

UBS Verifications · Decision Engine · July 2026

SNH *A Wright Original* · Built by **SNH Strategic Initiatives**

Most automation starts with an action: send an email, place a call, move the case. The Verifications Decision Engine starts earlier, with the decision that determines whether any action makes sense.

Each morning, the specialist model fleet reads every open search and estimates its likely path. The Case Control Engine names the case's current state, removes moves that no longer fit, uses the forecasts to rank what remains, and produces one next move. When the evidence changes, the plan changes with it.

Historical replay tested that complete handoff across 2.75 million case-days. It produced zero do-not-contact and hard-policy violations. It also found roughly

400,000 case-days when a response was already on record but another outreach attempt remained available. The important discovery was not simply that a rule could stop an unnecessary touch. It was that the operation needed a system capable of recognizing what had changed before choosing what should happen next.

The same discipline applies to the models. Three proposed improvements raised average accuracy but weakened the first page of the worklist, where the ranking matters most. The promotion gate rejected all three. A model is not better merely because its overall score improves. It must improve the part of the operation where its ranking will actually be used.

From evidence to execution in five steps

The architecture has five layers, and the arrows matter more than the boxes.

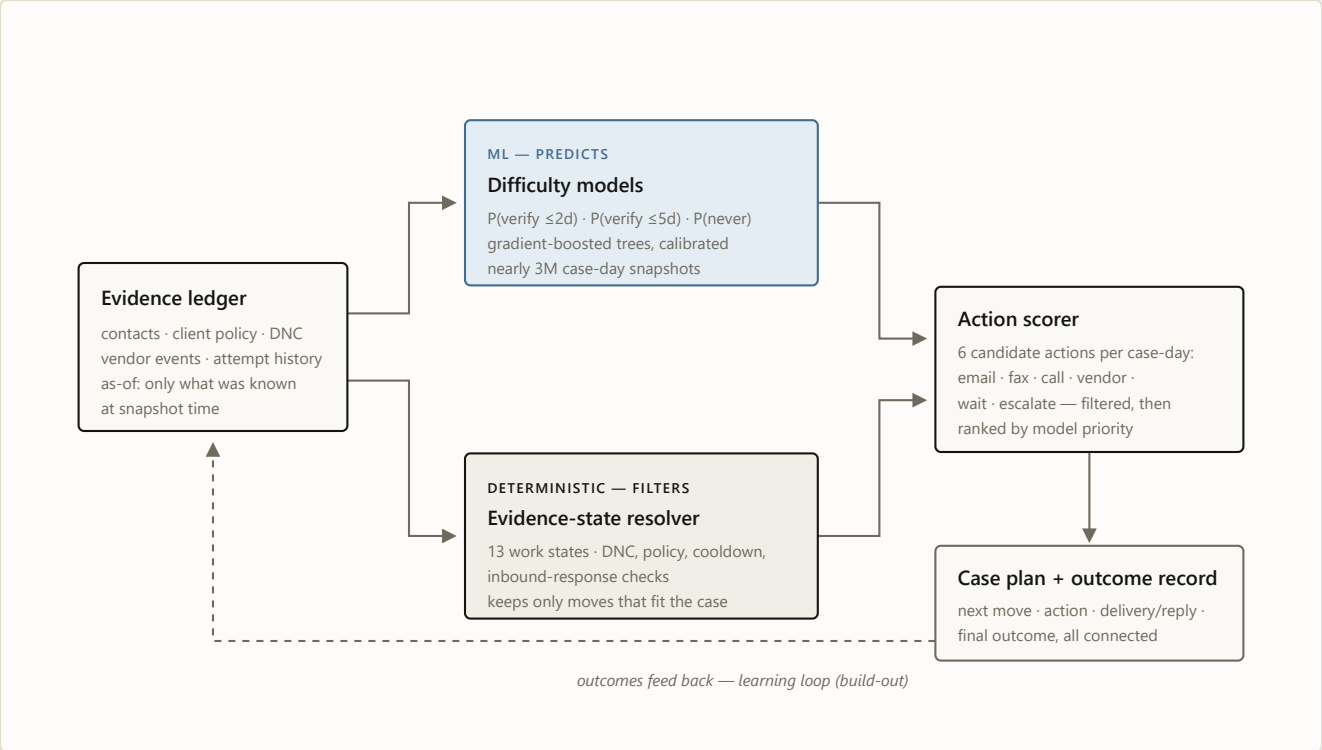


FIGURE 1 The decision path. The model fleet forecasts the case. The resolver removes moves that do not fit its current state. The scorer ranks what remains. The dashed loop — learning from logged outcomes — is the build-out, not the current system.

The load-bearing rule sits in the middle: **the resolver alone defines which actions exist. Models rank only the actions that survive.** A blocked action is removed, not penalized, so no level of model confidence can restore it.

The resolver, action scorer, router, and outcome record together form the **Case Control Engine**. It turns the model fleet's forecasts into an ordered case plan and is built to automate routine decisions one proven lane at a time.

One case-day, from evidence to action

Here is the whole handoff on a single day, using the illustrated guide's fictional case 4127. On day three, the case enters as a morning snapshot. The machine-

learning models estimate its likely path. The resolver names it outreach-ready. The rules remove fax, because no fax number exists, and remove escalation, because the case has not reached that point. The scorer ranks the four remaining moves and places email first. The Case Control Engine produces email as the next best move and connects it to the eventual outcome.

The models never rank a flat menu of actions. The Case Control Engine first removes the moves that do not fit the case, then uses the model scores to order what remains. That is the same sequence built to move from recommendation to automatic execution as each routine lane proves itself.

What the specialist model fleet sees

The specialist fleet does not make one broad prediction. Different models answer different questions: which new cases are likely to verify, which open cases are beginning to stall, and which are likely to end unable to verify. Every specialist reads the same morning snapshot — one per open case per day, nearly three million across a twelve-month window of employment, education, reference, and license-style checks — and sees only what was knowable that morning. I chose gradient-boosted trees because this evidence is structured, the relationships are uneven, and the system needs probabilities that can be tested and calibrated, not merely labels.

Table 1. The specialist model fleet (champions v3, promoted 2026-07-02). Scores are ROC-AUC: May 2026 is the hol never-seen snapshots; June 2026 is a further out-of-time month scored with frozen bundles. Every model is calibrate under the same as-of discipline, and passes the leakage audit; champions additionally clear the promotion gate.

SPECIALIST	OPERATIONAL QUESTION	VALIDATED RESULT	STAT

SPECIALIST	OPERATIONAL QUESTION	VALIDATED RESULT	STAT
Never verifies ends "unable to verify" (UTV) — the headline target	Will this case ever verify?	0.706 May 2.40× top-decile lift; June: champion held, lift 2.61× vs 2.47×	char
Verifies within 2 days	Will it resolve within two days?	0.747 May · 0.745 June 2.52× top-decile lift	char
Verifies within 5 days	Will it resolve within five days?	0.717 May · 0.726 June 1.81× top-decile lift	char
Day-0 intake read	How hard is this case, at arrival?	≈0.81 walk-forward, three forward months	ider
Day-1/2 re-score	Is the case beginning to stall?	0.73 / 0.70 holdout; lifts the aging employment-check read 0.62 → 0.71	supp
Early warning	Will it strand by day 20?	0.776 single-month holdout	prel

In plain terms, the never-verify specialist places the eventual failure ahead of the eventual success **71% of the time**, versus 50% for chance. The two-day and five-day specialists sort at 75% and 72%. At intake, when the system compares a case that will eventually fail with one that will verify, it ranks the future failure as harder approximately **81% of the time**. Operationally, the result that matters is the spread: the easiest predicted tenth fails 3% of the time, while the hardest fails 48% of the time.

FIGURE 2 Observed unable-to-verify rate by predicted-risk decile, May 2026 holdout. The model separates cases that fail 3% of the time from cases that fail 48% of the time. That spread is what makes a *fair* UTV metric possible: misses can finally be measured against what was actually winnable.

That 16× spread is also the business story. Today's UTV rate treats cases with radically different odds as though they were equally winnable. Difficulty-adjusted measures separate the easy, hard, and recoverable middle so management can aim effort and judge outcomes more fairly.

A high score still cannot break a rule

The models estimate what is likely. The resolver identifies what has already happened, what is blocked, and which moves still fit the case. The two jobs stay separate because they fail differently: a bad ranking wastes effort; a bad filter can create an incident. The resolver classifies every case-day into one of thirteen work states — terminal, already-responded, policy-blocked, awaiting response, cooldown, review-due, research-needed, digital-paths-exhausted, standard-paths-exhausted, outreach-ready, and so on — and builds the set of viable moves from the state, never from a score. Here is where the 2.75 million snapshots actually sit:

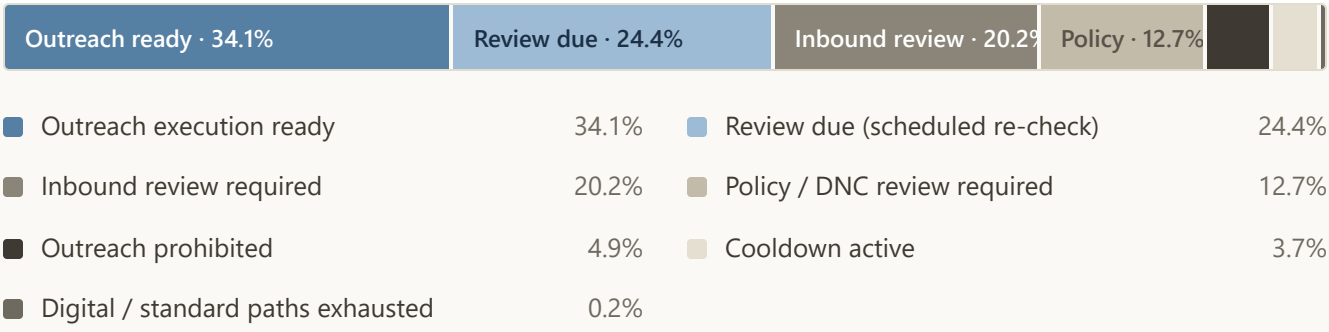


FIGURE 3 Resolver state mix across nearly three million snapshots. The interesting segment is the third one: cases where a vendor response is already on record but the legacy flow would have kept contacting.

I expected the next win here to come from a better model. It came from a rule — the **400,000-case-day finding** this post opened with, hiding in that third segment. The resolver stops those already-responded case-days from advancing to another touch. No model was needed — only the definition, written down. (The signal is a vendor receipt, not parsed reply content, and it does not exist for every product — the honest version of this number lives in the repo docs alongside its caveats.)

The same rule also shows why the safety result holds under pressure. Suppose a model strongly favors another call on one of those cases. The resolver sees that a reply is already on record, so call is removed before the final ranking. The model's confidence cannot restore it. That is why the replay produced zero do-not-contact and hard-policy violations: the rule checks run before the ranking, outside the models.

Downstream of the resolver, the scorer builds six candidate actions per case-day — email, fax, call, online vendor, wait, escalate — removes the moves that do not fit the current state, and ranks the rest by their calibrated scores. Across the full replay, that produced 16.5 million candidate actions.

A seven-stage pipeline validates every handoff from candidates through the final queue; every stage checks its own output and halts the run on any failure, and a strict contract validator signs off at the end. In the completed run, no prohibited action reached rank one, every case group kept an available move, and the final queue passed end to end.

No peeking, by construction

The easiest way for a model like this to look brilliant is to let information from after the outcome leak into its features — post-outcome status fields, completion timestamps, identity proxies. Every feature here is built under strict "as-of" discipline instead: 117 columns, each computed with strictly-before semantics. An automated leakage audit re-checks that boundary on every new training matrix — and again at every monthly retrain. The scores in this post are conservative by design, measured the hard, honest way.

The system rejected three of my improvements

Model improvements are promoted per target — there is no single "champion model," there are three — and promotion is mechanical. A challenger must clear every criterion against the sitting incumbent; a single checked-in champion config is the source of truth, and rejections are machine-recorded in the champion manifest.

ROC-AUC	at least +0.002 over the incumbent, with bootstrap confidence intervals excluding zero

Brier score	no worse than the incumbent — calibration is not negotiable
Top-decile lift	zero drop tolerated — the first page is where the ranking matters most
Capture at top 10%	zero drop tolerated
Slice gate	per-product and per-day slices, zero-tolerance with a reviewed, unit-tested allowlist for false positives
Out-of-time replication	the win must survive a never-seen month scored with frozen bundles

The gate earns its keep on what it rejects. Three times this year I tried to improve the never-verify model — a settings-tuning pass, a guarded version of the same sweep, and this cycle's note-reading variant — and the gate turned all three away **for the same reason**: each won on average and weakened the top of the ranked worklist, where accuracy is worth the most. One rejection is bad luck; three identical ones is a message. For this target, average accuracy and top-of-list sharpness genuinely pull against each other, so the next attempt changes the training data itself rather than pushing a fourth model variant.

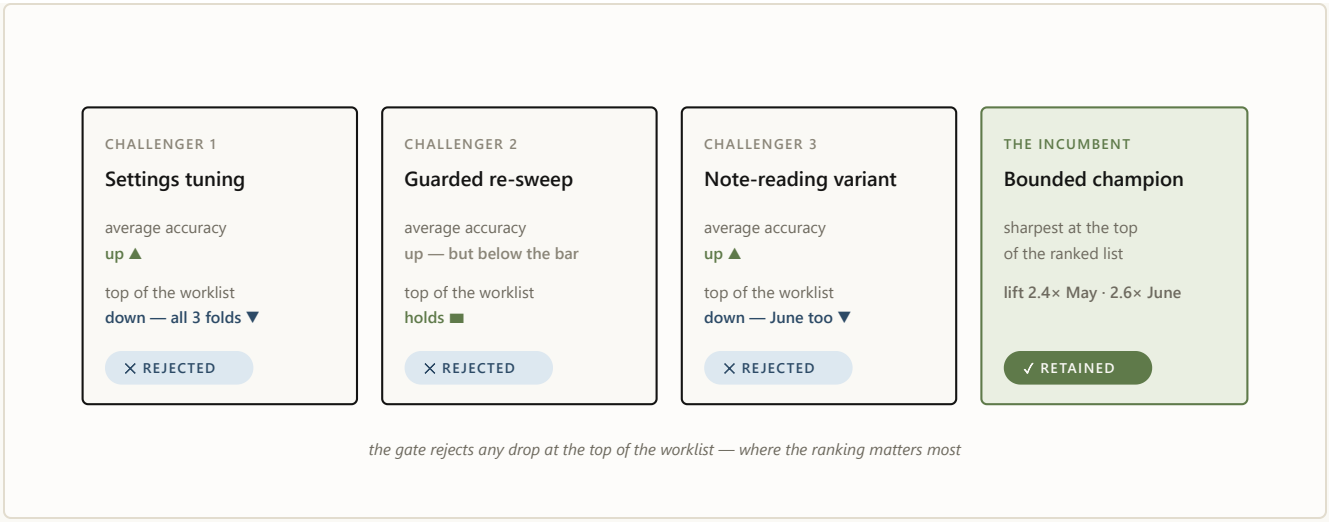


FIGURE 4 The gate at work on the never-verify target: three challengers, three different levers, one identical failure mode. Each verdict is machine-recorded — the rejections are as documented as the promotions.

Reading the notes without keeping them

The biggest accuracy gain this cycle came from the data source I had been most careful to avoid: the free-text notes attached to case history (“left a voicemail”; “reached HR, call back Tuesday”). Notes are where the richest signal lives — and where the risk lives twice over: outcome language (“verified via…”) leaks answers into training, and raw text carries the privacy exposure, so persisting it into training artifacts was never on the table.

The design that threads the needle: **a deterministic reading layer classifies each note the moment it streams past — then throws the text away**. Every note is sorted into eleven kinds of operational signal (contact made, voicemail left, callback promised, redirected to a third party, …), and only the signal survives: which kinds of events a case has seen, how recently, how often. Deterministic matters here — the same note always produces the same labels, so the privacy boundary has no black box in it and the whole layer can be audited and replayed. That turns 17.9 million

unreadable notes into 30 new machine-usable signals per case-day for the learned models, computed under the same no-peeking time discipline as everything else; about 28% of case-days carry at least one.

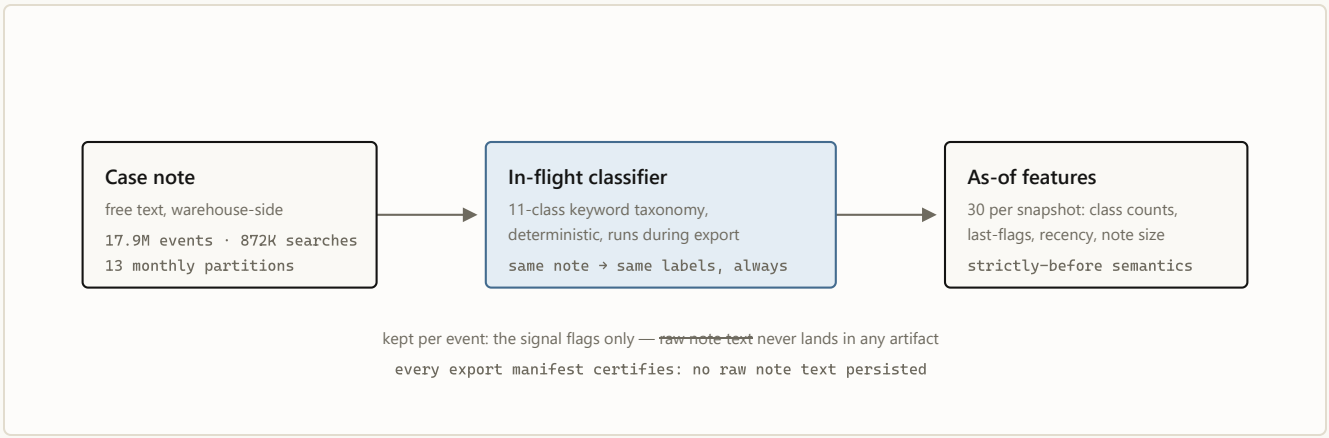


FIGURE 5 The inbound text layer. Signal crosses the boundary; text does not.

The same new evidence improved the two success forecasts but weakened the top of the never-verify worklist. Two challengers advanced; the third was rejected.

One operational note survives from that build: a routine row-count check against the export manifests caught a corrupted export *before anything consumed it*. Cheap invariant checks between pipeline stages are the highest-ROI code in the system.

What the evidence still will not let me say

The layers ahead are ambitious, which makes the boundary between built, ready, and not-yet-proved more important — not less. The repo maintains an explicit disallowed-claims list, and it does more for the project's credibility than any AUC. The current lines:

- **Historical replay is not live performance.** The replay establishes that the system can read, route, and structure the work at scale. Live execution, cycle time, and operational outcomes remain to be measured.
- **No causal claims about channel, wording, or templates.** Historical patterns like “email-first cases verified more often than call-first cases” are observational. Causal language stays banned until the live record connects the next move, action, template version, delivery or reply, and final outcome. The logging structure is built; the evidence is not yet collected.
- **Nothing auto-sends today.** Contact research, message drafting, automated routine decisions, and controlled execution remain future capabilities. Each advances only on its own logged results.

From prediction to automated decisions

Historical replay can show which cases look harder and whether the Case Control Engine produces a workable next move. It cannot prove that one channel, message, or follow-up caused a better outcome. That requires a live record connecting the case, the next move, the action, and the outcome.

The system advances through one repeatable sequence: read the case, build the plan, learn from live outcomes, automate the routine decision, then connect that decision to execution.

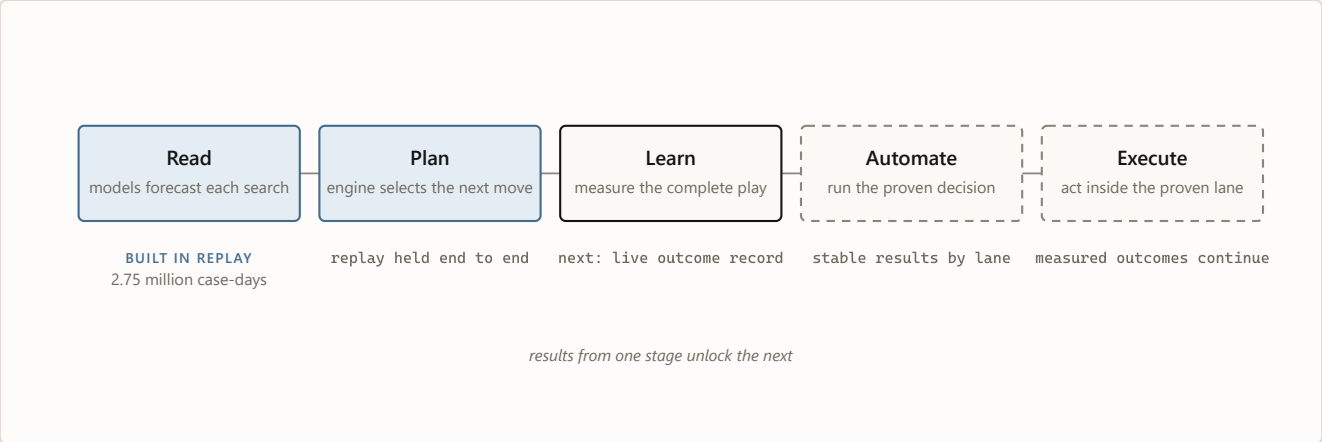


FIGURE 6 The operating sequence. Read and Plan are built in historical replay. Live integration creates the outcome record. That record identifies the first routine decisions ready for automation and controlled execution.

The live outcome record is the next engineering dependency. It connects each next move to the action taken, the channel and timing used, the message delivered, the reply received, and the final outcome. That record starts the next handoff:

- **The live record captures the complete chain:** next move, action taken, channel, timing, message, delivery, reply, and final outcome. That complete chain — not the final outcome alone — is what teaches the system which play worked.
- **The play engine learns the complete play:** channel, timing, message family, follow-up, and fallback — from pilot data that records which options were offered, not just which was taken.
- **The contact researcher repairs the inputs:** fresh, verified emails, fax numbers, and phone numbers enter as new evidence, so unworkable cases can become workable before attempts are wasted.
- **The SLM drafts the outreach** — last, because it is graded last: only the pilot's causal instrumentation can tell a well-worded email from a lucky one.

- **The Case Control Engine automates what proves itself:** a narrow lane moves first to automated routine decisions and then to controlled execution only on its own logged record — one lane at a time, never from a roadmap date.

The architecture can extend to new customers, but the learning stays customer-aware. The same evidence structure, specialist-model design, and promotion process can be reused while predictions, policies, cadence, and language are recalibrated against each customer's own outcomes. Learning should cross customer boundaries only where the evidence shows that the behavior transfers reliably.

Each layer receives its inputs from the layer before it. The resolver supplies the state. The model fleet supplies the forecast. The Case Control Engine builds the plan. The live record supplies the evidence needed to improve and automate the next decision.

The system today is deliberately disciplined: calibrated probabilities, deterministic checks, mechanical promotion gates, and a written list of things I refuse to say. Every component can change; the sequence persists. Evidence enters. Models forecast. The Case Control Engine builds the plan and selects the next move. Outcomes teach. Today that move is a recommendation. As each lane proves itself, the same sequence can execute routine decisions automatically. Starting narrow is how the automation becomes reliable enough to expand. Restraint is the launch mechanism, not the destination. ◆

The series. This is the engineering deep dive, including the path from prediction to automated decisions. Also: the illustrated walkthrough (one case through the machine) · the executive briefing · the Murphy Brown field report.

SNH *A Wright Original* · Built by **SNH Strategic Initiatives**

UBS Verifications · Decision Engine · July 2026

AFTER THE FOUR PIECES

The operation we are building

Six machine-learning models already read every open case and predict its path; the Case Control Engine turns those forecasts into a plan; a person reviews the next move. That claim rests on 2.75 million replayed case-days and zero policy violations — not on a destination slide.

The live workflow records what the system recommended, what a person chose, what was sent, who replied, and how the case ended. That record teaches the models which complete plays work, sends an AI researcher after better contact paths, and gives a small language model the evidence to draft the outreach. As a narrow lane proves itself, the Case Control Engine can automate its routine decisions and advance it to controlled execution while exceptions stay with people.

The first release recommends. The destination learns.

The engine begins by recommending the next move. It ends as an operation that learns how to complete the case.